vague-sw-p8-vague-shared

An AgentMPI run analysis

Abstract

Eight agents were each given one module of a small SQL engine and a specification that deliberately withholds every module's signatures, so that the RMA interface window was the only channel through which a rank could learn what its dependencies expose. They published eight interfaces into the window, fenced, read the fourteen interfaces the module dependency graph requires, wrote 2,201 lines of Python, and passed 60 of 60 acceptance cases at the first integration. That first-round success is the reason this cell has the fewest events of the four in the 2×2 — 311 against 473–686 — because the harness breaks out of its repair loop on a green build, so the run simply stopped after one round while its siblings each ran two. The run is therefore not a story of ranks doing less work; it is a story of not having to do the second half — though only one of the three siblings ran its second round for an honest reason, because the two `real-sw` cells were produced before the oracle's report parser was fixed and were told that trees which in fact score 60/60 and 59/60 had failed to import. Against its matched sibling `vague-sw-p8-vague-noshared`, which differs only in that the window is withheld, it used half the agent calls (16 against 32), 36% of the output tokens (32,304 against 88,989) and 39% of the money ($0.65 against $1.64) to reach the same 60/60. The single most important caveat is that "interface agreement" here means publish-once and read-once: no slot ever reached version 2, no lock was ever taken, and one of the eight published interfaces was factually wrong about a peer's type name and was never corrected.

1 What this run is

property	value
run	`vague-sw-p8-vague-shared`
experiment	`software`
campaign label	`vague-sw`
declared world size	8
ranks appearing in the log	8
executors	`broker` × 8, `worker` × 23
events	311
wall time	401.86 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	4.36×	total busy time over wall time
parallel efficiency	54.5%	achieved parallelism per rank
idle fraction	45.5%	rank-seconds paid for and unused
peak ranks busy	8	of 8 in the world
serial fraction of active time	15.4%	time with exactly one rank busy
load imbalance	1.47×	slowest rank's busy time over the mean
coordination share	44.8%	rank-seconds blocked in collectives, of those available
coordination in flight	83.1%	wall clock with any rank inside a collective
rank reattachments	23	fresh agent processes that took over a rank role
agent calls	16	invocations that reached a model
agent latency p50 / p95	82.03 / 257.58 s	per invocation
messages	52	45 eager, 7 rendezvous
tokens sent	43,079	31,415 deferred under rendezvous
tokens in / out	54,169 / 32,304	prompt and completion
cost	$0.6471	0.0200 $/kTok out

3 What the analysis notices

- [ok] 23 rank reattachment(s) occurred, up to incarnation 4: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 6 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

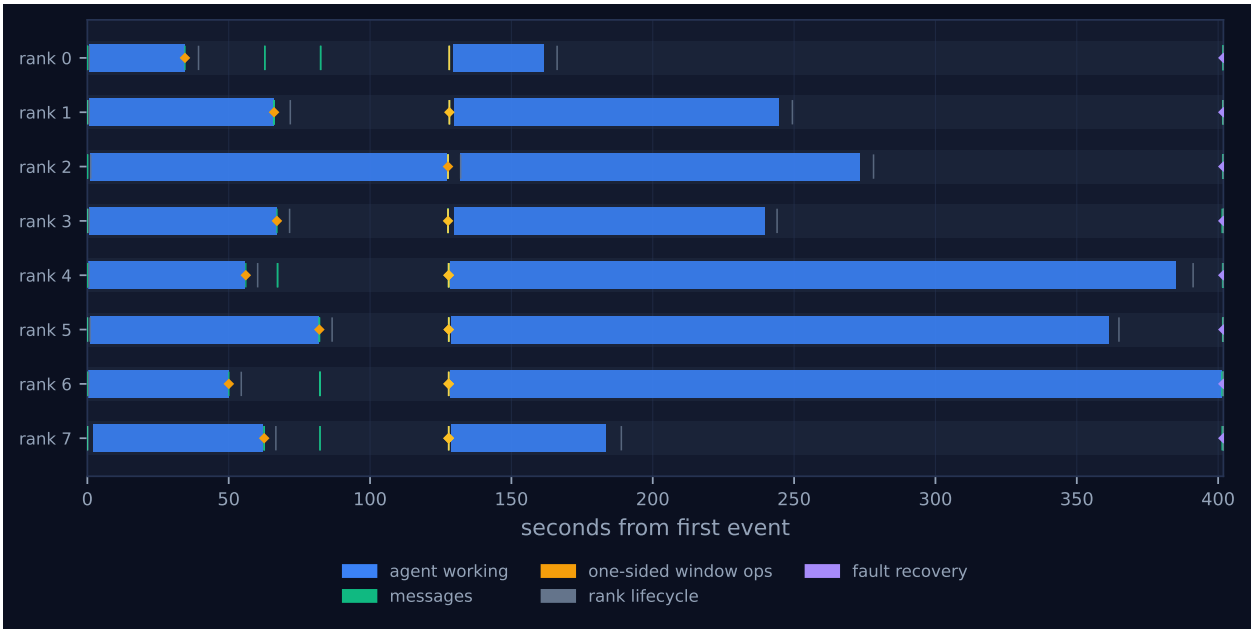

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

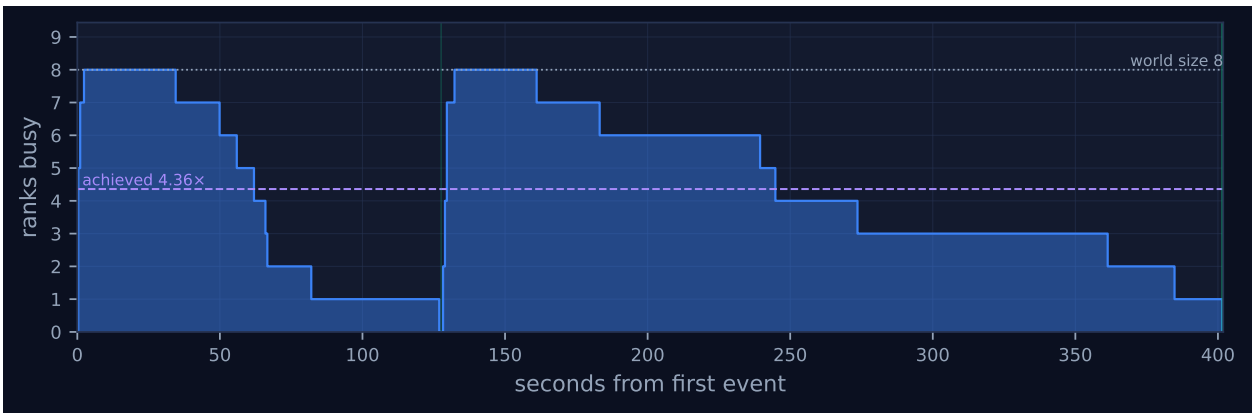

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

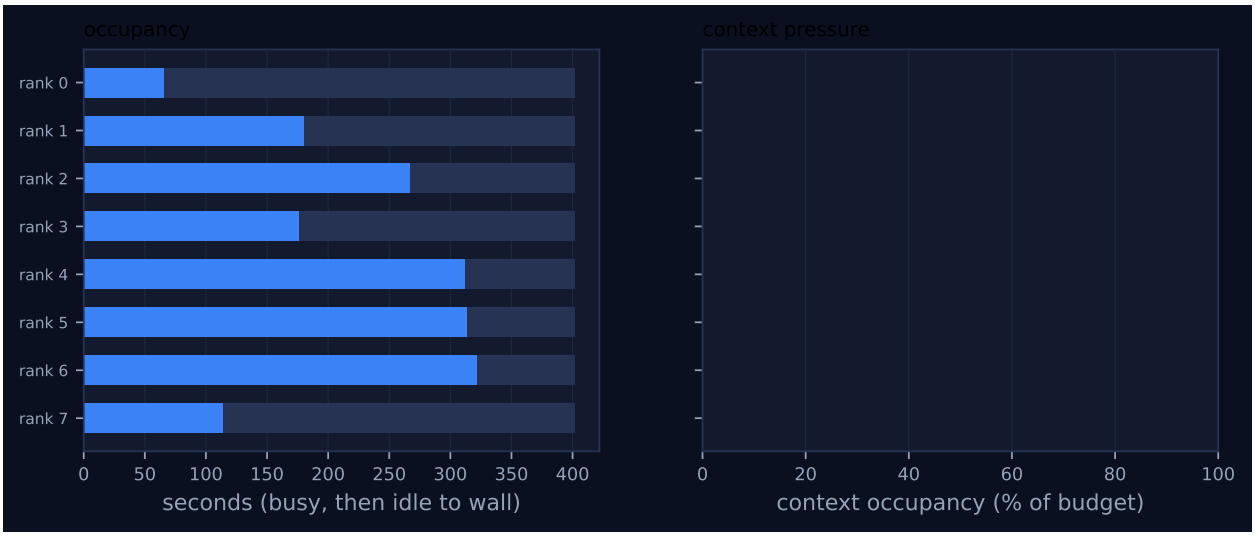

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

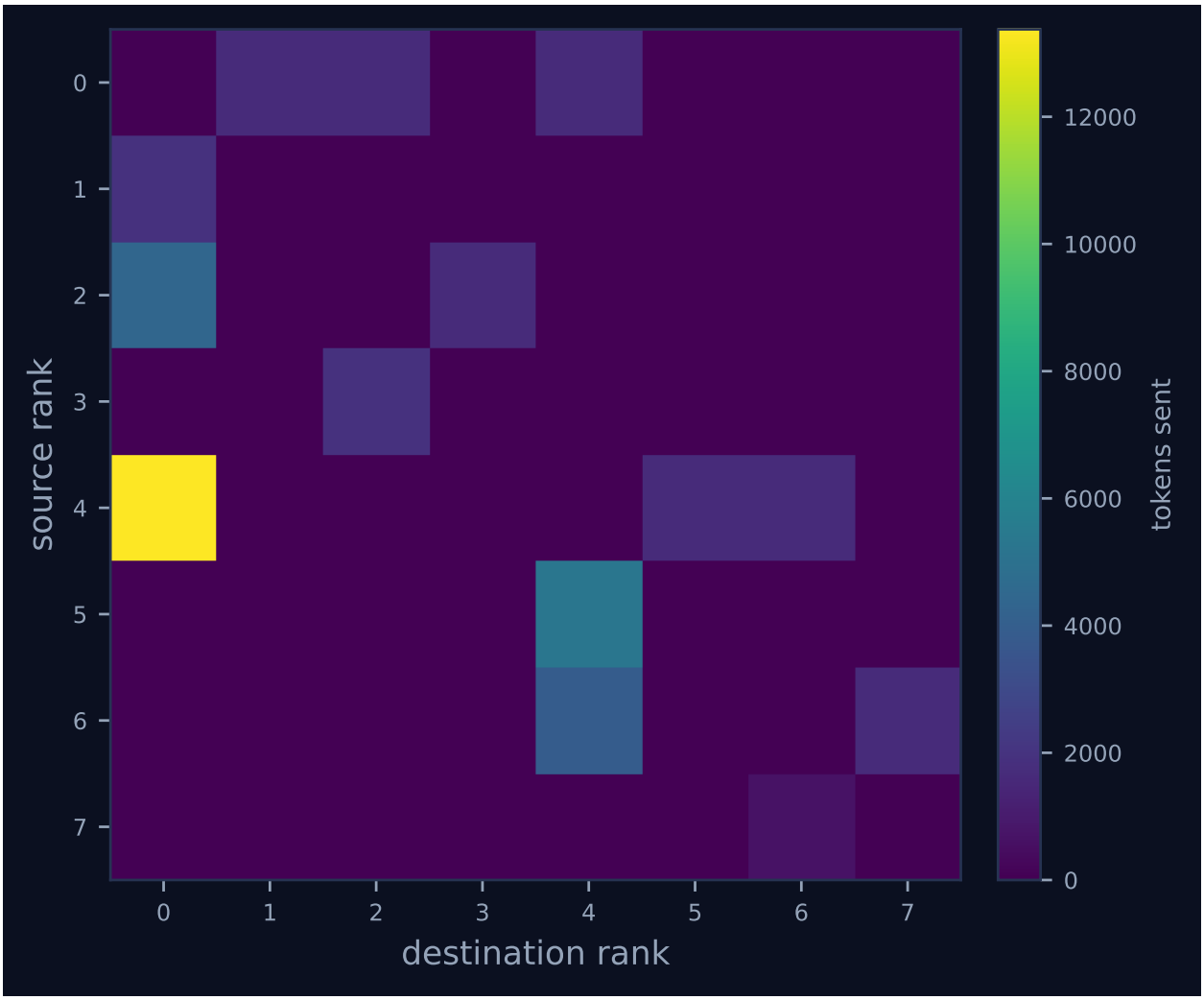

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

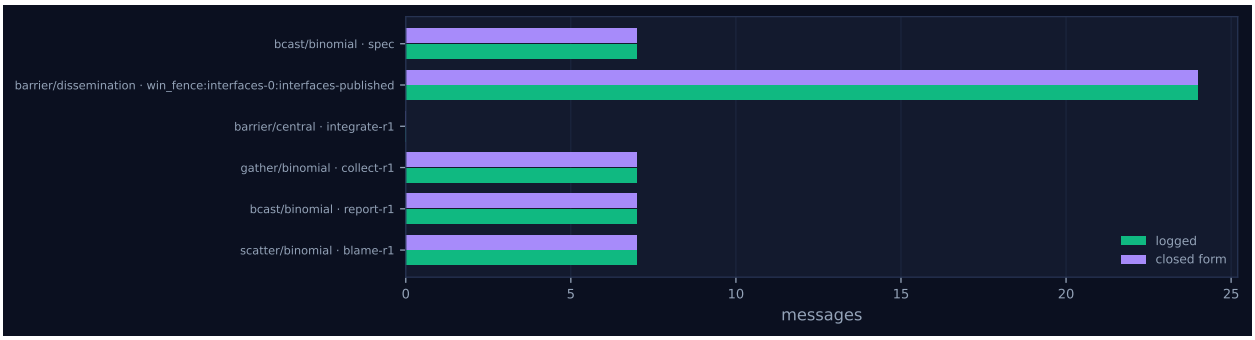

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	65.84	16.4	2	2	12	6	4,996	0.0	finalized
1	180.21	44.8	2	2	4	6	1,907	0.0	finalized
2	267.28	66.5	2	2	7	7	6,067	0.0	finalized
3	176.14	43.8	2	2	4	6	1,911	0.0	finalized
4	311.88	77.6	2	2	10	8	16,702	0.0	finalized
5	313.49	78.0	2	2	4	6	5,253	0.0	finalized
6	322.48	80.3	2	2	7	7	5,564	0.0	finalized
7	114.58	28.5	2	2	4	6	679	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)
bcast	binomial	spec	8	8	3	3	7	7	10,787	0.03
barrier	dissemination	win_fence:interfaces-0:interfaces-published	8	8	0	3	24	24	24	93.50
barrier	central	integrate-r1	8	8	0	2	0	0	0	240.08
gather	binomial	collect-r1	8	8	3	3	7	7	31,495	0.05
bcast	binomial	report-r1	8	8	3	3	7	7	749	0.43
scatter	binomial	blame-r1	8	8	3	3	7	7	24	0.00

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 shows two bars per rank and nothing else, and that is the whole run. The first bar is the interface call, the second is the implementation call, and the vertical alignment of every second bar at $t \approx 127.6\,\mathrm{s}$ is the fence. This is bulk-synchronous execution in its purest form: two phases, each ending when its slowest member ends.

The phase boundary is set by rank 2, which owns `nodes`. Its interface call ran for 127.54 s and emitted 1,939 output tokens, the largest interface in the run and the longest single call of the first phase; the fastest, rank 0's `errors`, finished at 34.51 s. The seven other ranks therefore sat inside `win_fence:interfaces-0:interfaces-published` waiting for it, and the barrier records how long each waited: 93.51 s for rank 0, 77.76 s for rank 6, down to 45.76 s for rank 5, summing to 476.6 rank-seconds of idle. The concurrency trace in Figure 2 shows this as the first staircase, falling from eight busy ranks to one and staying at one for the 45.28 s between 81.83 s and 127.11 s when rank 2 alone is working. The second phase repeats the shape exactly: `impl:engine:r1` on rank 6 runs

for 273.58 s, everyone else arrives early at `integrate-r1` and waits — 240.08 s for rank 0, 218.06 s for rank 7, 960.9 rank-seconds in total — and the last 16.48 s of the run has rank 6 as the only busy rank. Those two spans, 45.28 s and 16.48 s, are the entire 15.4% serial fraction reported in the headline table; there is no third contributor.

Adding the two straggler times, $127.57 + 273.58 = 401.15$ s, accounts for 401.15 of the 401.86 s of wall time. Everything the protocol did — the specification broadcast, the fence's own propagation, the gather of every module's source to rank 0, the acceptance run, the report broadcast, the scatter of blame and the agreement vote — fits in the remaining second. The oracle itself, the only real computation in the run, occupies 58.9 ms between rank 0's `coll.gather` at 401.6435 s and its `report-r1` broadcast at 401.7024 s: it wrote nine files, launched a subprocess and ran 60 SQL cases in 0.015% of the run.

The two coordination figures in the headline table have to be read together, and this run is an unusually clean illustration of why they are now reported separately. The rank-second share is 44.8%: of the $8 \times 401.86 = 3{,}214.9$ rank-seconds the job paid for, 1,439.3 were spent inside a collective call. The span share is 83.1%: for 333.9 of the 401.9 seconds of wall clock, at least one rank was inside a collective. The first says under half the population's time went on coordination; the second says that for five-sixths of the run somebody somewhere was blocked. Both are true and they answer different questions. The coincidence in this run — its span share is numerically identical to the rank-second share the metric used to report before the proportion was corrected — is instructive rather than accidental: what the old figure was computing was collective wall over run wall, which is a span, so on a run like this one it happened to land on the right answer to the wrong question. In the corrected 2×2 the four cells sit at 41.7% (`real-sw-p8-full`), 56.4% (`real-sw-p8-noshared`), 44.8% (this run) and 51.5% (`vague-sw-p8-vague-noshared`), a much tighter and much less alarming spread than the old numbers suggested.

Neither figure should be read as coordination overhead. Both count time spent inside a collective call, and in this run essentially all of that time is a rank blocked at a barrier while a peer talks to a model. The collectives cost nothing; the waiting costs everything, and the waiting is a consequence of agent latency dispersion, not of the protocol.

The fence looks expensive in rank-seconds and is nearly free in makespan. Reconstructing from the log what a dependency-triggered start would have given — each module beginning as soon as the last of its declared dependencies had been `put`, with the measured implementation latencies held fixed — yields a finish at 385.15 s against the actual 401.40 s, a saving of 16.25 s or 4.0%. Only `engine` was genuinely over-constrained, ready at 82.06 s and started at 127.81 s; but `parser`, the second-longest implementation, depends on `nodes` and could not have started before 127.57 s in any case. The last publisher was already on the critical path, so the global barrier cost almost nothing beyond what the dependency structure cost anyway. That is a property of this particular schedule and not of fences in general, and it is worth saying because the 476.6 rank-seconds of fence idle is the number a reader will notice first.

Figure 4 is the binomial gather tree and nothing else. Rank 4 is the bright cell at 13,375 tokens because it aggregates ranks 5 and 6 before forwarding to rank 0; the `collect-r1` gather moves 31,495 tokens, 73% of the 43,079 tokens sent in the entire run, in the seven rendezvous messages that are the run's only non-eager traffic. The right-hand panel of Figure 3 is empty, and that is by construction: the harness passes `admit=False` on every collective, so all eight ranks finalise with `context_used` = 0 and this run exercises no admission behaviour at all. Its context metrics are vacuous, not encouraging.

7 Interpretation

Why the event count is low. The counter-intuitive fact about this cell is that it has the fewest events of the four despite the hardest specification. The trace answers it unambiguously and the answer is not that ranks did less: there are 16 `agent.call` events, two per rank, and each rank made exactly one interface call and one implementation call, all 16 succeeding on `attempt: 1` with zero contract violations, zero `harness.degraded` events, zero failed ranks and eight `rank.finalize` events. What the run did not do is a second round. The harness loops up to `rounds` = 2 and breaks when `agree` returns true; the eight `ft.agree` events between 401.83 s and 401.86 s all carry `result: true` with all eight ranks as voters and none excluded, because the acceptance suite reported 60 of 60 passing on the first integration. Everything downstream of that vote — the second implementation call per rank, the peer-review `neighbor_allgather` pair, the `topo.graph_create` calls, the second gather, bcast, scatter and agree, and the renegotiation `put` under a lock — is absent from this trace because the loop exited. Its three siblings each ran two rounds and each has 32 agent calls. The 311 events are the cost of getting it right the first time.

What the window actually carried. There were exactly 8 `win.put` events, one per module slot, all writing version 1 over version 0, all with `locked: false` and `stale_write: false`; one `fence`; and exactly 14 `win.get` events. Fourteen is the number of edges in the declared module dependency graph (tokens 1 + functions 1 + parser 3 + planner 2 + engine 3 + api 4), so every rank read every dependency it declared and no rank read anything else. Every one of the fourteen returned `version: 1` with a non-zero token count, so not one read fell through to the `default={}` the harness supplies; `errors` was read six times, and 15,374 tokens were read out of the 9,946 tokens published, a fan-out of 1.55×.

That the reads were used, and not merely performed, can be checked in the delivered source rather than inferred. Parsing the round-1 tree, there are 42 names imported across module boundaries and all 42 resolve to a definition in the module they were imported from. `parser` alone imports 17 names from `nodes` — `SelectStmt`, `JoinClause`, `OrderItem`, `SelectItem`, `TableRef`, `Star`, `IsNull`, `InList`, `Like`, `UnaryOp`, `BinaryOp`, `FuncCall`, `ColumnRef`, `Literal`, `Expr`, `AGGREGATE_FUNCTIONS` and `SCALAR_FUNCTIONS` — every one of which appears verbatim in the 1,939-token interface that `nodes` published, and none of which appears anywhere in the vague specification. There are zero `getattr` or `hasattr` lookups across a peer-module boundary in the whole tree. The window is the only place those names could have come from.

What it did not do. No slot ever reached version 2. The renegotiation branch is guarded on `my_failures` being non-empty, and no rank had any failures, so `Window.critical` was never entered: the contention report gives `n_locks: 0`, `n_contended: 0`, `max_lock_wait_s: 0.0` and there is not a single `win.lock` or `win.unlock` event in the log. The zero stale writes reported for this run are therefore true but empty — `Window.put` raises a staleness violation only when the prior version was greater than zero, and every write here was the first write to a virgin slot, so a lost update was structurally impossible. This run says nothing whatever about the locking discipline or the staleness detector. What it establishes about the window is narrower and cleaner than "negotiation works": one exposure epoch, one access epoch, publish once, read once, and it was enough.

The exposure epoch cannot converge mutually referential interfaces. All eight interfaces are written inside the same exposure epoch, so no rank can see any other rank's interface while writing its own. An interface that mentions a peer's type is therefore a guess, and one of the eight guessed wrong. `parser` published, at 56.04 s, `def parse(sql: str) -> Select`, with a note reading "the root node of the tree published by `minidb.nodes` (the `Select` node type)". `nodes` published at 127.57 s and its root is `SelectStmt`; the name `Select` does not occur anywhere in the delivered `nodes.py`. After the fence `parser` read that 1,939-token interface and implemented `def parse(sql: str) -> SelectStmt`, quietly correcting itself. Nothing was broken, because `api` — which read `parser`'s interface at 127.81 s and depends on it — binds the result to `statement: Any` and never names the type. But the record in slot `parser` is still version 1 and is still wrong, and it stayed wrong precisely because the run succeeded: the only mechanism the harness has for correcting a published interface runs when a round fails. A consumer that had trusted the published return type would have been misled, and no part of AgentMPI would have flagged it — the record is not stale, it is simply false, and `win.get` reports version and token count, not truth. This is a defect of the experiment's single-epoch design rather than of the window implementation, but it identifies a missing facility: there is no check that an implementation honours the interface its own rank published, and this run contains one violation out of eight in the delivered code.

The pairwise comparison, and what it supports. `vague-sw-p8-vague-noshared` is the right comparator and an unusually tight one: it is the next step of the same campaign, at the same commit `90ca469`, started fourteen minutes later against the same worker pool, with `shared_interfaces` the only differing flag. The interface phase is identical in both: 8 calls, 13,862 prompt tokens, 9,946 against 9,959 output tokens, and — checking result digests in the two fabrics — six of the eight interface responses are byte-identical across the two runs. The divergence is entirely downstream. With the window, 8 implementation calls emitting 22,358 tokens produced a tree that passed 60/60 and ended the run. Without it, 8 implementation calls emitting 33,764 tokens passed 55/60, which bought a peer-review round (8 more calls, 32,208 prompt tokens for 7,108 of findings) and a second implementation round (8 more calls, 38,158 tokens) before reaching 60/60. Totals: 16 against 32 agent calls, 32,304 against 88,989 output tokens, \$0.6471 against \$1.6427, and 401.86 against 847.50 s.

The mechanism visible in the artefacts is more interesting than the totals. Denied a channel to learn its neighbours' exports, the no-window population wrote defensively rather than wrongly. Its final tree has 6 cross-module imported names against this run's 42; `parser` imports nothing at all from `nodes` or `tokens`, `planner` imports nothing from `nodes`, `api` imports whole modules and resolves entry points with `getattr` at call time, and `engine` probes its peers with five `getattr(_functions, ..., None)` lookups. Counting only `engine` and `api`, there are ten such dynamic lookups into peer modules in that tree and there are none anywhere in this one, and the no-window tree is 3,562 lines against this run's 2,201 — 62% more code for the same score. Per implementation call it emitted 4,495 output tokens against 2,795 here. That is what the window bought: not correctness, which both runs eventually reached, but the ability to name a peer's exports instead of defending against not knowing them.

One run per cell supports the direction of that difference and not its magnitude. It is a single draw, the two runs are not independent given the shared interface responses, and a 2.5× cost ratio from $n = 1$ per cell should be read as "roughly a repair round's worth", which is what it structurally is.

The flagged findings. Both are expected for this configuration. There were three until the coordination metric was corrected: at 83.1% the analysis raised a note that the run "spent more time coordinating than working", and at the corrected 44.8% it no longer does, which is the right outcome — summed over all ranks, the two barriers account for 1,437.44 s of the 1,439.28 rank-seconds of collective time and every other collective in the run for 1.84 s between them, and neither barrier took as much as a further second to resolve after its last participant arrived. What that note was really detecting was straggler dispersion, and it is better read off the imbalance figure of 1.47× and the per-rank busy times, which range from 65.8 s on rank 0 to 322.5 s on rank 6. Of the two findings that remain, the 23 reattachments to incarnation 4 are the broker executor's normal worker cycling and not a fault — but see below, because the claim that the reattachments preserved "their context accounts" is not quite true, and seven of the 23 attached after their rank's last agent call and claimed nothing. The 6-of-6 collective model agreement is correct and weak: five of the six invocations are seven-message binomial trees whose prediction is trivial, and only the 24-message dissemination barrier is a real check. This run also reports `n_misreported_collectives: 0` where all three siblings report 2, but that is because it never invoked the neighbourhood collectives at all, not because it is cleaner.

Two accounting defects, one since fixed and one still live. The first was that `tokens_deferred` was reported with two different definitions that disagreed by 2.4× in this run. `cost.summarise()` counted only the tokens of `msg.send` events in rendezvous mode and yielded 31,415, which is what `metrics.json` and the trace viewer's stats row displayed. `Job.totals()` summed a per-rank counter that `comm.py` incremented both on a rendezvous send and again on *every* receive taken with `admit=False`, and yielded 74,494, which is what the `job.finish` event and the results file recorded. The identity was exact and held in all four software runs: 31,415 rendezvous-send tokens plus 43,079 non-admitted-receive tokens equals 74,494, and since every receive in this harness is non-admitted, the second term was simply every token sent, counted a second time — so a reader taking the figure from the results file got 2.4× the figure on the same run's dashboard, against a `summarise` docstring promising that its numbers were "not an artefact of two different summarisers".

That is now fixed, and fixed in the way that keeps both quantities rather than discarding one. `tokens_deferred` is send-side and rendezvous-only, a subset of `tokens_sent` by construction, and measures transport mode; `tokens_unadmitted` is receive-side and transport-agnostic, and measures context admission. `cost.summarise` derives both from the log and `Job.totals()` reports both, so the collision is gone. This run's regenerated summary reads `tokens_deferred: 31415` and `tokens_unadmitted: 43079`, and the second number is now meaningful in its own right: 43,079 is every token the run sent, and every one of them was received without being admitted to a context, because the harness passes `admit=False` on every collective. The old conflated 74,494 was not a quantity anybody wanted.

The second defect is still live at HEAD, and I checked rather than assumed. A reattaching worker silently overwrites the rank's configured context budget. In this run the job was launched with `context_budget=120_000`, the eight incarnation-1 `rank.init` events say `budget: 120000`, every one of the 23 later `rank.init` events says `budget: 128000`, and the `ranks` table in the fabric ends the run holding `context_budget = 128000` for all eight ranks. The cause is that `cmd_worker` constructs its `RankRuntime` without passing `context_budget`, so it takes `DEFAULT_CONTEXT_BUDGET = 128_000`, and the `UPDATE` branch of `register()` writes that value back over the persisted one; the sibling code path `_comm()` reads the persisted budget from the `ranks` row first and does not have the bug, so the fix is one argument. Reproducing it against a scratch fabric at HEAD — create

a job, register rank 0 with a budget of 120,000, then re-register it exactly as `cmd_worker` does — the persisted budget still goes 120,000 to 128,000 and the two `rank.init` events still disagree. A related but distinct hardening has landed in the meantime: `register()` now refuses a rank index outside the job's world size, where it previously accepted any index unconditionally. That closes a different hole — a shared worker pool leaking rank rows into the wrong fabric — and does not touch the budget path, which runs after the range check and is unchanged. Nothing in this run depended on the budget, because nothing was ever admitted, but a run that relied on admission pressure would find its budget raised by 6.7% at the first reattachment, and the generated finding's claim that reattachment preserves the rank's context account remains false of the budget half of that account.

8 Threats to this reading

Half of the 2×2 *was driven by a false failure signal, and the rows are not commensurable on outcome.* Both `real-sw-p8-full` and `real-sw-p8-noshared` report `importable: false`, `n_total: 0` and 0/0 final acceptance for both of their rounds. That is not a failure of those populations; it is a failure of the harness's report parser. Their results files record commit `26ae006`, which predates `parse_report`; at that commit `run_acceptance` parsed the oracle's stdout with `json.loads(out[out.rfind("{"):])`, which lands inside a nested per-case object and never parses, and the stored `import_error` for `real-sw-p8-full` is visibly a truncated JSON report full of `"passed": true` cases.

The trees those runs left on disk settle it. Replaying the unmodified oracle (`experiments/minidb/acceptance.py`) against copies of them, `real-sw-p8-full` scores `importable: true` and 60/60 in both round 1 and round 2, and `real-sw-p8-noshared` scores `importable: true` and 59/60 in both, failing only `empty_table_select`. Two other analysts obtained the same figures independently. So the precise-spec population composed a working SQL engine at first integration and was told it had not compiled. Worse than wasted, the repair round was misdirected: because an unparseable report is treated as an import failure, the harness broadcasts a synthetic `IMPORT` case to every rank instead of scattering real blame, so the `noshared` population never saw its one genuine failure — and `empty_table_select` is still failing at the end of round 2.

The consequence for the 2×2 has to be stated plainly, and this document is the place to state it, because this cell is the one whose *event count* the comparison leans on. Comparisons between the `real-sw` row and the `vague-sw` row can legitimately use token counts, code size, message counts and window traffic, because those measure what the populations did and are unaffected by how the oracle's output was parsed. They cannot use round count, and they cannot use "did it pass". Two of the four cells ran their second round because of a parsing bug rather than because of anything in their code, and both of those cells report a pass rate of zero for a tree that passes at least 59 of 60. The headline of my own run — that it has the fewest events of the four because it needed one round rather than two — is therefore sound as a statement about this cell and about its matched sibling `vague-sw-p8-vague-noshared`, which was run at the same post-fix commit, and is not sound as a statement about the `real-sw` pair, whose round count was set by the bug. Nothing here touches this run, which is at commit `90ca469`, after the fix, and whose 60/60 was parsed correctly and confirmed by the agreement vote.

Wall time is the least reliable of the comparisons. The six interface responses that are byte-identical between this run and the no-window run took very different times to produce: `iface:nodes` emitted the same 1,939 tokens in 127.54 s here and 29.01 s there, 15.2 against 66.8 tokens per second for

literally the same output. Median throughput over all calls is 18.1 tok/s in this run against 28.9 tok/s in the sibling. Service rate varied by roughly 3× between two runs fourteen minutes apart, so the 2.11× wall-time ratio is mostly noise. It happens to be conservative — the run that won on wall time was the one served more slowly — but the defensible comparisons are the ones that count work rather than time: agent calls, output tokens, dollars, and rounds. Relatedly, `latency_s` in this trace is quantised to 0.5 s because that is `BrokerExecutor.poll_s`; all 16 values are multiples of half a second, and they should not be read to millisecond precision.

The counterfactual makespan is a reconstruction, not a measurement. The 385.15 s figure holds implementation latencies fixed while moving start times earlier, which assumes the worker pool would have served eight overlapping calls at the same rate — plausible here, since eight were already served concurrently in each phase, but it is an assumption and the 16.25 s saving is an estimate.

The success may be over-determined. 60 of 60 at the first integration is a ceiling, and a ceiling cannot show how much headroom the window has. The vague layout still names each module's responsibility and its permitted imports, and `minidb` is a well-known shape of program; a capable model can guess a great deal of the boundary from "the abstract syntax tree representation" alone. The no-window run reaching 55/60 unaided, and 60/60 after one repair round, is the measure of how much: the window's contribution here is real but it is the last five cases, not the first fifty-five. On a system whose interfaces were less guessable the gap would be larger, and on one more guessable it would vanish — which is exactly what happened to the precise-spec version of this ablation, and why the vague layout exists.

Two of the run's headline properties are true by construction and should not be read as results. Zero stale writes follows from every put being a first write, as argued above. Zero context pressure follows from `admit=False` on every collective. And `n_recovery: 8` in `metrics.json`, rendered as "fault recovery" in the viewer legend, counts the eight `ft.agree` votes, which the trace vocabulary classifies as recovery events; no fault occurred in this run, no rank was excluded from the vote, and the number should be read as "one agreement, eight participants".